



CHILD COMPONENTS IN ANGULAR



`codewithrutu_09`

✓ **What Are Nested Components?**

Nested (child) components are components declared inside another component's template.

 **Think: Parent-child relationship.**

The parent passes data to the child, and the child can emit events back.

 **Helps create reusable, maintainable UI pieces.**

✓ Creating a Child Component

You can create one using Angular CLI:



```
ng generate component child
```

This creates:



```
child
├── child.component.ts
├── child.component.html
└── ...
```

Angular auto-declares it in the parent module.

✓ Using a Child Component in a Parent

Parent component template:



```
<app-child></app-child>
```

👉 This renders the **ChildComponent** inside the **ParentComponent**.

↺ You can place it multiple times with different inputs.

✓ Passing Data to Child via @Input()

Child:



```
@Input() username: string;
```

Parent Template:



```
<app-child [username]="parentName"></app-child>
```

↻ Binds parentName (parent) to
username (child)

📌 Use @Input() to receive data from
the parent.

✓ Emitting Events from Child using @Output()

Child:



```
@Output() greet = new EventEmitter<string>();

sendGreeting() {
  this.greet.emit('Hello from child!');
}
```

Parent Template:



```
<app-child (greet)="handleGreet($event)"></app-child>
```

📢 Use @Output() to send data/events from child → parent

✓ Use Case Example – Product List

Parent Component:

ProductListComponent

- Passes each product to child

Child Component:

ProductCardComponent

- Receives product via @Input()
- Emits “Add to Cart” via @Output()

Promotes reusability

Keeps logic and UI responsibilities separate

✓ Nested Component Best Practices

🧠 Keep your components:

- Small & focused
- Reusable
- Well-named
- Decoupled

Use @Input() for passing down state

Use @Output() for upward communication

📌 Avoid deeply nested trees unless necessary